

```

"""
Local Coding Agent platform for Qwen2.5-Coder- GPU Optimized (PC & Colab
Compatible)
"""

from __future__ import annotations

import argparse
import importlib
import importlib.util
import os
import re
import subprocess
import sys
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Callable, List, Optional, Tuple

# -----
# Dependency bootstrap
# -----

def _has_flag(flag: str) -> bool:
    return flag in sys.argv

def _install(packages: List[str]) -> None:
    if not packages:
        return
    print("Installing missing packages:", ", ".join(packages))
    subprocess.check_call([sys.executable, "-m", "pip", "install", "--upgrade",
*packages])

def ensure_dependencies() -> None:
    if not torch.cuda.is_available():
        return

    core = [
        ("transformers", "transformers"),
        ("accelerate", "accelerate"),
        ("bitsandbytes", "bitsandbytes"),
    ]
    missing = [pkg for mod, pkg in core if importlib.util.find_spec(mod) is None]
    if missing:
        _install(missing)
        os.environ["CODING_AGENT_RESTARTED"] = "1"
        os.execv(sys.executable, [sys.executable] + sys.argv)

import torch
ensure_dependencies()

```

```

from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig

MODEL_ID = "Qwen/Qwen2.5-Coder-7B-Instruct"
MAX_NEW_TOKENS = 3072

def load_model(log: Callable[[str], None] = print):
    if not torch.cuda.is_available():
        raise RuntimeError("GPU NOT DETECTED! Please enable T4 GPU.")

    log(f"GPU Detected: {torch.cuda.get_device_name(0)}")
    log(f>Loading model: {MODEL_ID} (This may take a few minutes...)")

    tokenizer = AutoTokenizer.from_pretrained(MODEL_ID, trust_remote_code=True)

    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True,
    )

    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID,
        quantization_config=bnb_config,
        device_map="auto",
        trust_remote_code=True
    )
    return tokenizer, model

_CACHE = {}

def generate(prompt: str):
    if "model" not in _CACHE:
        _CACHE["tokenizer"], _CACHE["model"] = load_model()

    tokenizer, model = _CACHE["tokenizer"], _CACHE["model"]

    # Refined system message to ensure Python code output
    messages = [
        {"role": "system", "content": "You are an expert Python developer. Provide clean, efficient, and well-commented Python code for the user's request."},
        {"role": "user", "content": prompt}
    ]

    inputs = tokenizer.apply_chat_template(messages, tokenize=True,
add_generation_prompt=True, return_tensors="pt", return_dict=True).to(model.device)

    with torch.no_grad():
        output = model.generate(**inputs, max_new_tokens=MAX_NEW_TOKENS,
pad_token_id=tokenizer.eos_token_id)

```

```
    return tokenizer.decode(output[0, inputs.input_ids.shape[1]:],
skip_special_tokens=True)

if __name__ == "__main__":
    try:
        # Interactive loop for requesting custom Python code
        user_task = input("Describe the Python code you want me to generate: ")
        if user_task.strip():
            print(f"\nGenerating Python code for: {user_task}")
            print("----" * 10)
            result = generate(user_task)
            print(result)
    except RuntimeError as e:
        print( e)
```